COMPUTE

AWS provides a variety of compute services for running applications and processing data in the cloud. These services provide flexible computing resources without requiring organizations to purchase and maintain physical servers.

AWS compute services support different approaches, including virtual machines, containers, serverless computing, and managed application platforms. Selecting an appropriate compute service depends on factors such as application design, usage patterns, and the level of infrastructure control required.

Compute Services Overview

AWS compute services differ in how applications are deployed and in how responsibilities for the underlying computing environment are divided between AWS and the customer.

Types of AWS Compute Services

Virtual Machine-Based Computing
Amazon Elastic Compute Cloud (Amazon EC2) provides resizable virtual machines called instances. EC2 follows an Infrastructure as a Service (IaaS) approach in which AWS provides the underlying physical infrastructure while customers configure and manage their virtual computing environments.

This approach provides flexibility because customers can select computing resources according to the requirements of their workloads. It is useful when an application requires greater control over its operating system, software, networking, and other server configurations.

Serverless Computing
Serverless computing enables applications or functions to run without requiring customers to provision and manage servers. AWS manages the underlying infrastructure and provides the computing resources needed to execute the code.

AWS Lambda is an example of a serverless compute service. Lambda can run code in response to events or according to a schedule. This approach is useful for workloads that do not require a continuously running server and can reduce the operational work associated with managing computing infrastructure.

Container-Based Computing
Containers package an application together with the components it needs to run. They provide a portable and lightweight way to deploy applications across computing environments.

AWS provides several services for containerized workloads. Amazon Elastic Container Service (Amazon ECS) provides container orchestration, while Amazon Elastic Kubernetes Service (Amazon EKS) provides a managed Kubernetes environment. AWS Fargate provides serverless compute capacity for containers, reducing the need to manage the underlying servers. Amazon Elastic Container Registry (Amazon ECR) provides a managed service for storing and retrieving container images.

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI

Managed Application Platforms
AWS Elastic Beanstalk provides a managed platform for deploying and running web applications. Developers provide their application code while Elastic Beanstalk assists with infrastructure provisioning, capacity scaling, load balancing, and application health monitoring.

This approach reduces the infrastructure tasks developers need to perform and allows them to focus more on developing and maintaining their applications.

Amazon Elastic Compute Cloud

Amazon Elastic Compute Cloud (Amazon EC2) provides virtual servers, called instances, in the AWS Cloud. EC2 instances can be launched in different Availability Zones and configured with the computing, storage, networking, and security resources required by an application.

Customers have control over the guest operating system of an EC2 instance and can install and configure applications as needed. EC2 can support many workloads, including web servers, application servers, database servers, file servers, and other computing workloads.

Amazon Machine Images
An **Amazon Machine Image (AMI)** is a template used to launch an EC2 instance. It contains the information required to create the instance, such as the operating system and other software configurations included in the image.

AWS provides AMIs, and customers can also use images from other sources or create their own. Selecting an AMI is one of the first decisions when launching an EC2 instance because it determines the initial software environment of the virtual server.

EC2 Instance Types
An **instance type** determines the combination of computing resources available to an EC2 instance. Instance types provide different combinations of processor, memory, storage, and networking capacity to support different workload requirements.

AWS organizes instance types into families designed for different purposes. These include:

- **General purpose** – Provides a balance of compute, memory, and networking resources.
- **Compute optimized** – Designed for workloads that benefit from high processing performance.
- **Memory optimized** – Designed for workloads that process large amounts of data in memory.
- **Storage optimized** – Designed for workloads that require high performance when accessing data stored locally.

The appropriate instance type depends on the resource requirements of the workload.

Launching an EC2 Instance
Launching an EC2 instance requires several configuration decisions. These settings determine the instance's computing environment, connectivity, permissions, storage, and security.

The major decisions include:

1. Select an AMI – Choose the operating system and initial software configuration.

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI

2. Select an instance type – Choose the required compute, memory, storage, and networking capacity.
3. Configure network settings – Select the VPC, subnet, and other network settings for the instance.
4. Assign an IAM role – Provide the instance with appropriate permissions to access AWS services when required.
5. Provide user data – Supply commands or scripts that can run when the instance is launched.
6. Configure storage – Specify the storage volumes required by the instance.
7. Add tags – Assign metadata to help identify, organize, and manage the instance.
8. Configure a security group – Define the permitted inbound and outbound network traffic.
9. Select a key pair – Configure credentials that can be used to securely connect to supported instances.

These settings can be adjusted according to the purpose and requirements of the instance.

EC2 Instance Lifecycle

An EC2 instance moves through different states during its lifecycle. When an instance is launched, it enters the **pending** state while AWS prepares it. It then enters the running state and becomes available for use.

A running instance can be **stopped** when its computing capacity is temporarily unnecessary. A stopped instance can later be started again. When an instance is no longer needed, it can be **terminated**, which permanently deletes the instance.

Understanding these states is important because the state of an instance can affect resource availability and charges.

Elastic IP Addresses

An **Elastic IP address** is a static public IPv4 address that can be associated with supported AWS resources, including EC2 instances. Unlike an automatically assigned public IPv4 address, an Elastic IP address remains allocated to the AWS account until it is released.

An Elastic IP address can be reassociated with another supported resource when necessary. This can help maintain a consistent public address when the underlying instance changes. AWS charges for public IPv4 addresses, including Elastic IP addresses, according to current AWS public IPv4 pricing.

Instance Metadata

EC2 provides instance metadata, which contains information about a running instance. Applications or scripts running on the instance can access metadata to obtain information about the instance and its environment without requiring that information to be manually stored on the instance.

Metadata can include information related to the instance, network interfaces, and other instance attributes.

Monitoring Amazon EC2

Amazon EC2 integrates with Amazon CloudWatch to monitor instances and collect performance metrics. These metrics can provide information about the operation and resource utilization of an instance.

Monitoring helps identify changes in workload behavior and provides information that can be used when evaluating instance performance and capacity. This information can also support later decisions about scaling and cost optimization.

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI

Amazon EC2 Cost Optimization

Amazon EC2 costs can be managed by matching computing resources and purchasing options to actual workload requirements. Cost optimization involves selecting appropriate instance resources, adjusting capacity as demand changes, choosing suitable pricing options, and reviewing storage use.

Right-Sizing EC2 Instances

Right-sizing involves selecting an EC2 instance type and size that match the performance requirements of a workload. An instance that provides more resources than necessary can result in unnecessary costs, while an undersized instance can affect application performance.

Workload performance should be monitored to determine whether the selected instance continues to provide the appropriate amount of compute, memory, storage, and networking resources. As workload requirements change, an instance can be moved to a more appropriate size or type.

Increasing Elasticity

Elasticity allows computing capacity to adjust according to workload demand. Instead of maintaining enough capacity for peak usage at all times, resources can be added when demand increases and reduced when demand decreases.

For workloads that do not need to run continuously, EC2 instances can also be stopped when they are not required and started again when needed. **Amazon EC2 Auto Scaling** can automatically adjust the number of instances according to defined conditions, helping applications maintain appropriate capacity as demand changes.

Selecting an EC2 Pricing Option

AWS provides different EC2 pricing options for different workload patterns and commitments.

- **On-Demand Instances** – Provide computing capacity without a long-term commitment. They are suitable for workloads with changing or unpredictable requirements.
- **Savings Plans** – Provide reduced prices in exchange for a commitment to a consistent amount of compute usage for a one- or three-year term.
- **Reserved Instances** – Provide a billing discount for eligible EC2 instance usage in exchange for a one- or three-year commitment. They can be appropriate for predictable workloads.
- **Spot Instances** – Use spare EC2 capacity at discounted prices. AWS can interrupt Spot Instances when the capacity is needed, so they are best suited to workloads that can tolerate interruptions.
- **Dedicated Hosts** – Provide a physical server dedicated to a single customer and can support workloads with specific compliance or software licensing requirements.

Selecting a pricing option should be based on factors such as workload duration, predictability, flexibility, and tolerance for interruption.

Optimizing Storage

Storage attached to EC2 instances also contributes to workload costs. Storage should therefore be selected according to the capacity and performance requirements of the application.

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI

Unused storage resources and unnecessary snapshots should be reviewed regularly. Storage volumes can also be adjusted as requirements change so that workloads do not continually maintain more storage capacity or performance than necessary.

Measure, Monitor, and Improve
EC2 cost optimization is an ongoing process. Workload usage and performance should be monitored to determine whether resources continue to match actual requirements.

Services and tools such as **Amazon CloudWatch** can provide metrics for evaluating resource utilization. The information collected can be used to identify opportunities to resize resources, adjust capacity, or reconsider the pricing and storage options being used.

EC2 cost optimization can therefore focus on four areas: right-sizing resources, increasing elasticity, selecting appropriate pricing options, and optimizing storage choices.

Container Services

Containers provide a way to package an application together with the libraries, dependencies, and configuration it needs to run. This allows an application to operate consistently across different computing environments and makes it easier to deploy and scale application components.

Unlike virtual machines, containers share the operating system of the host rather than requiring a separate guest operating system for each application. As a result, containers are generally lightweight and can start more quickly than virtual machines.

Docker Containers
Docker is a software platform commonly used to build, package, and run containerized applications. A Docker container is created from a **container image**, which contains the application code and the components required for the application to run.

Multiple containers can run on the same host while remaining isolated from one another. This makes containers useful for applications composed of independent components that need to be deployed or scaled separately.

Amazon Elastic Container Service
Amazon Elastic Container Service (Amazon ECS) is a managed container orchestration service for deploying, managing, and scaling containerized applications. ECS coordinates where containers run and helps maintain the desired number of running containers.

With Amazon ECS, containers can run using two primary compute options:

- Amazon EC2 – Containers run on EC2 instances managed as part of the container environment. This option provides greater control over the underlying compute resources.
- AWS Fargate – Containers run without requiring customers to provision or manage the underlying servers. Customers specify the resources needed by their containers, while AWS manages the compute infrastructure.

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI

Amazon Elastic Kubernetes Service

Kubernetes is an open-source system for deploying, scaling, and managing containerized applications. It organizes containers into groups and coordinates their operation across computing resources.

Amazon Elastic Kubernetes Service (Amazon EKS) is a managed Kubernetes service that enables organizations to run Kubernetes applications on AWS. AWS manages the Kubernetes control plane, reducing the operational effort required to maintain the underlying Kubernetes infrastructure.

Amazon EKS can run container workloads using Amazon EC2 or AWS Fargate, depending on the level of infrastructure management and control required.

Amazon Elastic Container Registry

Amazon Elastic Container Registry (Amazon ECR) is a managed container image registry. It provides a location where container images can be stored, managed, and retrieved for deployment.

For example, an application can be packaged as a container image and stored in Amazon ECR. Amazon ECS or Amazon EKS can then use that image when deploying containers.

Introduction to AWS Lambda

AWS Lambda is a serverless compute service that runs code without requiring customers to provision or manage servers. Developers provide the code and configure when it should run, while AWS manages the underlying compute infrastructure needed to execute it.

Lambda is designed for applications in which code runs in response to specific events or requests. Instead of maintaining a continuously running server, a Lambda function runs when it is invoked and can automatically scale as the number of requests changes.

Lambda Functions

Code that runs in AWS Lambda is organized into Lambda functions. A function contains the code that performs a particular task together with configuration settings that determine how the function operates.
When creating a Lambda function, settings can include the runtime environment, memory allocation, timeout, permissions, and other configuration needed by the function. AWS provides runtime environments for several commonly used programming languages.

Event-Driven Computing

AWS Lambda is commonly used for event-driven computing, where an event causes a function to run. Events can originate from other AWS services or applications.

For example, a Lambda function can be invoked when:

- An object is uploaded to an Amazon S3 bucket.
- Data is added or changed in an Amazon DynamoDB table.
- A request is received through an application.
- A scheduled event occurs.

This allows applications to perform tasks automatically when particular activities occur.

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI

How AWS Lambda Works

The basic operation of AWS Lambda can be described as follows:

1. An event occurs or a request invokes a Lambda function.
2. AWS Lambda allocates the resources required to run the function.
3. The function executes the configured code.
4. Lambda automatically manages the underlying compute environment.
5. Resources can scale according to the number of incoming events or requests.

Because the underlying servers are managed by AWS, customers can focus on the function code and its configuration rather than server administration.

Benefits of AWS Lambda

AWS Lambda provides several benefits for workloads that are suitable for serverless computing:

- **No server management** – AWS manages the infrastructure used to run the functions.
- **Automatic scaling** – Lambda scales in response to incoming requests or events.
- **Pay for use** – Charges are based on requests and the compute resources consumed while functions execute.
- **Integration with AWS services** – Functions can respond to events generated by other AWS services.
- **Flexible resource configuration** – Resources such as memory and execution settings can be configured according to the needs of a function.

AWS Lambda Use Cases

Lambda can support tasks that need to run automatically when an event occurs. For example, when an image is uploaded to an Amazon S3 bucket, an event can invoke a Lambda function that creates a thumbnail version of the image.

Lambda can also automate operational tasks. A function can be invoked according to a schedule to perform actions such as starting or stopping Amazon EC2 instances. These examples demonstrate how Lambda can connect events with application or administrative actions without requiring a continuously running server.

Introduction to AWS Elastic Beanstalk

AWS Elastic Beanstalk is a service for deploying and managing web applications in the AWS Cloud. Developers upload their application code, and Elastic Beanstalk handles supporting tasks such as resource provisioning, load balancing, scaling, and application health monitoring.

Elastic Beanstalk provides a managed approach to application deployment while still allowing access to the AWS resources that support the application. This enables developers to focus on application development while AWS assists with managing the deployment environment.

Elastic Beanstalk Environments

When an application is deployed through Elastic Beanstalk, the service creates an environment that contains the AWS resources required to run the application. Depending on the application configuration, these resources can include Amazon EC2 instances, load balancing, and scaling resources.

Elastic Beanstalk monitors the environment and provides information about its health. Developers can also modify configuration settings when the application's requirements change.

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI

Supported Platforms

Elastic Beanstalk supports applications developed using several commonly used platforms and programming environments. These include:

- Docker
- Go
- Java
- .NET
- Node.js
- PHP
- Python
- Ruby

The selected platform provides the environment needed to run the application code.

How AWS Elastic Beanstalk Works

A typical Elastic Beanstalk deployment follows these steps:

1. Create an Elastic Beanstalk application.
2. Select the appropriate platform for the application.
3. Upload the application code.
4. Configure the application environment.
5. Elastic Beanstalk provisions the resources required to run the application.
6. Elastic Beanstalk monitors the deployed environment and supports scaling as required.

Developers can deploy updated versions of the application as the code changes without manually rebuilding the entire supporting environment.

Benefits of AWS Elastic Beanstalk

AWS Elastic Beanstalk provides several benefits for application deployment:

- **Simplified deployment** – Developers can deploy applications without manually configuring each supporting AWS resource.
- **Infrastructure management** – Elastic Beanstalk handles many provisioning and configuration tasks required by the application environment.
- **Scalability** – Application capacity can adjust as workload requirements change.
- **Health monitoring** – Elastic Beanstalk monitors the health of the application environment.
- **Developer control** – The underlying AWS resources remain accessible and can be configured when additional control is required.

References:

Amazon Web Services. (2022). *AWS Academy Cloud Foundations learner guide*. Amazon Web Services.

Amazon Web Services. (n.d.). *Amazon EC2 user guide*. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/

Amazon Web Services. (n.d.). *Amazon Elastic Container Service developer guide*. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI

Amazon Web Services. (n.d.). *Amazon Elastic Kubernetes Service user guide*. https://docs.aws.amazon.com/eks/latest/userguide/

Amazon Web Services. (n.d.). *AWS Lambda documentation*. https://docs.aws.amazon.com/lambda/

Amazon Web Services. (n.d.). *AWS Elastic Beanstalk developer guide*. https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/Welcome.html

PROPERTY OF STI
PROPERTY OF STI
PROPERTY OF STI